

A Column Generation Heuristic for the Two-Dimensional Two-Staged Guillotine Cutting Stock Problem with Multiple Stock Size.

Fabio Furini, Enrico Malaguti, Rosa Medina Durán,
Alfredo Persiani, Paolo Toth

DEIS, University of Bologna, Viale Risorgimento 2, 40136 Bologna, Italy
emails:{fabio.furini, enrico.malaguti, rosa.medinaduran, alfredo.persiani,
paolo.toth}@unibo.it

Abstract

We consider a Two-Dimensional Cutting Stock Problem where stock of different sizes is available, and a set of rectangular items has to be obtained through two-staged guillotine cuts. We propose a heuristic algorithm, based on column generation, which requires as subproblem the solution of a Two-Dimensional Knapsack Problem with two-staged guillotine cuts. A further contribution of the paper consists in the definition of a Mixed Integer Linear Programming Model for the solution of this Knapsack Problem, as well as a heuristic procedure based on dynamic programming. Computational experiments show the effectiveness of the proposed approach, which obtains very small optimality gaps and outperforms the heuristic algorithm proposed by Cintra et al. [3].

1 Introduction

Given a set of rectangular items and infinitely many identical rectangular bins, the *Two-Dimensional Cutting Stock Problem* (2DCSP) asks to cut all the items by using the minimum number of bins or, equivalently, by minimizing the global area of the used bins. We consider the Two-Dimensional *Two-Staged Guillotine Cutting Stock Problem with Multiple Stock Size* (MS2DCSP), that is, a 2DCSP where: i) All the items must be obtained through guillotine cuts, i.e., cuts that are parallel to the sides of the bin and cross the bin from one side to the other; ii) All the items must be obtained with two-staged cuts, where each stage consists in a set of parallel guillotine cuts performed on the shape obtained in the previous stage (we allow trimming, i.e., a third stage cut can be used to separate a rectangle from a waste area); and iii) Bins of different sizes are available. In the left of Figure 1 we give an example of two-stage guillotine cuts on a rectangular bin: first stage cuts (horizontal) are represented with a thick line, second stage cuts (vertical) with a thin line, and trimming cuts (horizontal) with a dotted

line. Waste area is represented by a dashed surface. In the right part of the figure we represent a cutting which cannot be obtained through guillotine cuts. We consider the two cases where the items can or cannot be orthogonally rotated. In the typology introduced by Wäscher et al. [13], the problem is the constrained version of the *Two-Dimensional Rectangular Multiple Stock Size Cutting Stock Problem*.

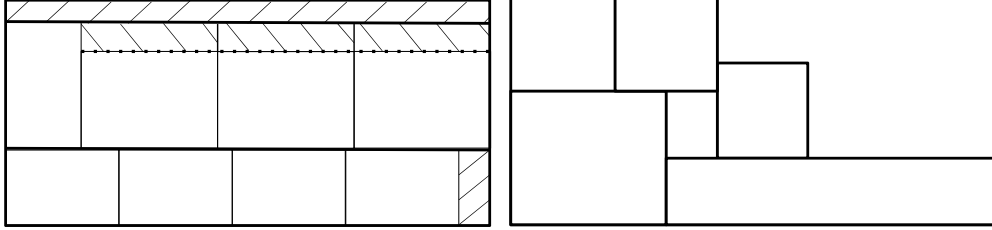


Figure 1: Guillotine cut (left) and non-guillotine cut (right).

Formally, we are given m classes of rectangular items, each item class i ($i = 1, \dots, m$) having a demand d_i , and n classes of rectangular bins, each bin class j ($j = 1, \dots, n$) having an infinite number of copies. Each item of class i ($i = 1, \dots, m$) has dimensions (l_i, w_i) , where l_i and w_i are, respectively, the length and the width of the item. Each bin of class j ($j = 1, \dots, n$) has dimensions (L_j, W_j) , where L_j and W_j are, respectively, the length and the width of the bin. All the input data are assumed to have integer positive values. We have to define the cutting patterns (simply denoted as patterns in the following) so as to obtain all the requested items in the specified demand, by minimizing the global area of the used bins. The problem is NP-hard and arises in the industry whenever a sheet of paper, wood, glass, or metal has to be cut.

2DCSP was introduced by Gilmore and Gomory [7], who as well introduced the k – staged version of the problem and considered bins of different sizes. Alvarez-Valdes et al. [1] developed and compared several heuristic methods, based on column generation, for solving the general Two-Dimensional Cutting Stock Problem. For solving the subproblems, in addition to dynamic programming, they developed some heuristics. Riehme et al. [12] considered the two-staged version of the problem in the case where bins of different sizes are available and the demands of the item classes differ in a large range. Cintra et al. [3] considered several Two-Dimensional Guillotine Cutting Stock Problems and their variants in which orthogonal rotations are allowed. They presented dynamic programming algorithms for the Rectangular Knapsack Problem, which are then used to generate patterns in approaches for the Cutting Stock Problems based on Column Generation. In particular, they considered the two-dimensional two-staged guillotine version of the Cutting Stock Problem where bins of different sizes are available and rotation of the items is allowed. Their algorithm generates columns to optimally solve the continuous relaxation of the Cutting Stock Model (see Section 2), which is then iteratively rounded-down so as to obtain an integer feasible solution. The algorithm by Cintra et al. [3] is the only algorithm in the literature reporting detailed computational results which allow a complete comparison. A survey on several Two-Dimensional Packing Problems can be found in Lodi et al. [8].

The paper is organized as follows: in Section 2 we review a classical Cutting Stock model, in Section 3 we describe a model and a heuristic algorithm for the generation of cutting patterns,

and in Section 4 we present the overall Column Generation Heuristic. Computational results are presented and discussed in Section 5, and Section 6 concludes the presentation.

2 Cutting Stock Model

Our approach is based on column generation and follows the seminal work by Gilmore and Gomory [5, 6] for the One-Dimensional Cutting Stock Problem. Let S_j ($j = 1, \dots, n$) be the family of all the feasible cutting patterns for bin class j , that is, cutting patterns which do not exceed the size of bin class j and can be obtained by two-staged guillotine cuts. Following Cintra et al. [3], we only allow patterns where the first cuts are horizontal. In addition, we consider only patterns which do not contain more items of an item class i than specified by the corresponding request d_i . MS2DCSP can be modelled by using an integer variable x_{s_j} associated with each feasible cutting pattern $s_j \in S_j$ for the bin class j ($j = 1, \dots, n$). The value of this variable denotes the number of times the pattern is used in the solution. The objective function minimizes the total area of the used bins. By introducing a parameter $C_{s_j}^i$ to represent the number of items of item class i ($i = 1, \dots, m$) in the cutting pattern s_j , and by defining $\kappa_j = L_j W_j$ as the area of a bin of bin class j , a classical model for MS2DCSP reads:

$$\min \quad z := \sum_{j=1}^n \sum_{s_j \in S_j} \kappa_j x_{s_j} \quad (1)$$

$$\text{subject to} \quad \sum_{j=1}^n \sum_{s_j \in S_j} C_{s_j}^i x_{s_j} \geq d_i \quad (i = 1, \dots, m) \quad (2)$$

$$x_{s_j} \in \mathbb{Z}^+ \quad (j = 1, \dots, n, s_j \in S_j) \quad (3)$$

By dropping the integrality requirement for the variables, imposed by constraints (3), we obtain the Linear Programming (LP) Relaxation of model (1)-(3), defined by (1), (2) and:

$$x_{s_j} \geq 0, \quad (j = 1, \dots, n, s_j \in S_j) \quad (4)$$

Model (1),(2) and (4), called *Master Problem* (MP), needs column generation techniques to be efficiently managed. Actually, it has exponentially many variables, which cannot be explicitly generated for large-size instances. Instead of the original Master Problem, a *Restricted Master Problem* (RMP) is considered which is initialized and solved with a subset of the exponentially many variables $x_{s_j}, s_j \in S_j, j = 1, \dots, n$. In particular, RMP is initialized with the variables (columns) corresponding to a feasible solution computed through the heuristic algorithm described in Section 3.1. Given a solution to RMP, new variables, needed to optimally solve MP, are obtained by separating the following dual constraints:

$$\sum_{i=1}^m C_{s_j}^i \pi_i^* \leq \kappa_j \quad (j = 1, \dots, n, s_j \in S_j) \quad (5)$$

where π_i^* ($i = 1, \dots, m$) represents the optimal dual variable associated with the i -th constraint of type (2). Accordingly, the so-called *Slave Problem* (SP) is to determine if, for some bin class j , a feasible pattern $s_j^* \in S_j$ exists, such that:

$$\sum_{i=1}^m C_{s_j^*}^i \pi_i^* > \kappa_j. \quad (6)$$

While the satisfaction of the demand associated with the item classes is imposed in MP, the SPs (one is defined for each bin class) are devoted to the generation of feasible cutting patterns satisfying constraints (6). SPs (see Section 3.2) consist in a series of Two-Dimensional Two-Staged Knapsack Problems with Guillotine Cuts (2DKP), where one looks for the pattern of maximum profit according to the current value of the optimal dual variables (profits) π_i^* ($i = 1, \dots, m$). If a feasible pattern s_j^* with maximum profit greater than κ_j is found, the column corresponding to s_j^* is added to the current RMP. When no feasible pattern with maximum profit greater than the corresponding κ_j is found, the rounded-up value of the optimal solution to MP is a valid lower bound for the original MS2DCSP.

3 Cutting pattern generation

A first approach to 2DKP was proposed by Gilmore and Gomory [7], and is based on two nested unbounded Knapsack Problems (KP). This is an exact method that generates cutting patterns which can be characterized by overproduction of items. Overproduction affects the value of the optimal solution of the continuous relaxation of the model, which can produce a weaker lower bound on the optimal solution value of the problem.

The best known compact optimization model to solve 2DKP was proposed by Lodi and Monaci [9]. The idea of this model (called M2 in the following) is to define a potential strip for each item to be cut, and to implicitly order the strips according to non-increasing widths of the associated items. A strip is initialized and can be used only if it contains the item whose width defines the strip width. This idea removes much of the symmetry which affects descriptive 2DKP models. Lodi and Monaci [9] proposed as well a model, called M1, more suited for Two-Dimensional Bin Packing Problems, where each item class contains a single item, i.e. $d_i = 1$ ($i = 1, \dots, m$).

3.1 Initial Heuristic

A heuristic algorithm for MS2DCSP is used to provide an initial feasible solution to the problem in a very short computing time. In order to take into account rotation, a copy of each item class is created by swapping length and width; then the new set of item classes is sorted according to non-increasing widths. In case more item classes have the same width, they are ordered by non increasing length. The algorithm associates each item class i with a bin class, say $b(i)$. $b(i)$ is defined as the bin class of minimum area such that a single bin of this class can fit the whole demand d_i of item class i . When no such bin class exists, $b(i)$ is defined as the bin class of

minimum area such that a single bin of this class can fit the largest number of items of class i .

The items are iteratively inserted into the bins by following the predefined order. The insertion is performed by strips: each strip determines a guillotine cut in the bin; the width of the strip equals the width of the first inserted item. Strips are filled from left to right, as explained in the following. Given an item of class i that has to be cut, we initialize a new strip, which is inserted in a partially cut bin or in a new bin of class $b(i)$ if such a bin does not exist. Consequently its demand d_i and the demand of the corresponding rotated item class are updated. When the strip cannot fit any additional item of class i , a new strip is initialized in the same bin or in a new bin of class $b(i)$ and the item is inserted in the new strip. Before proceeding with the new strip, a greedy procedure looks iteratively for items of the following item classes (with smaller width) that can fill the remaining space in the first strip. If such item class exists, the maximum number of items that can be cut according to the remaining demand d_i and to the remaining space are inserted. Consequently its demand d_i and the demand of the corresponding rotated item class are updated. The greedy procedure continues to fill the strip until no item can be inserted. The new strip is placed over the first strip in the current bin whenever it is possible. Otherwise, a new bin of class $b(i)$ is initialized and the greedy procedure explained before is used to fill the empty space over the strip. When the demand of the current item class i is satisfied, the algorithm continues with the next item class, in the order previous defined, that has a remaining demand. If such item class does not exist, the algorithm terminates.

3.2 A Mixed Integer Linear Model for the 2DKP

Given a bin class j ($j = 1, \dots, n$) and a feasible dual solution π_i^* ($i = 1, \dots, m$) to RMP, in order to generate new columns with negative reduced cost or to prove the optimality of the current master solution, we have to solve a 2DKP. In this section we propose a Mixed Integer Linear Programming (MILP) Model, called M_0 in the following. In the computational experiments (Section 5), M_0 will be compared with model M_2 proposed by Lodi and Monaci in [9]. We present the case where items can be rotated, which is more general; the case of no rotation trivially follows. For a specified bin class with dimensions (L, W) , define $I = \{i : (w_i \leq W \text{ and } l_i \leq L) \text{ or } (l_i \leq W \text{ and } w_i \leq L)\}$ as the index set of the item classes which can fit in the bin, and let ϱ be the maximum number of strips which can fit in the bin. This value can be computed following this procedure:

- set $a_i = \min \{l_i, w_i\}$, $i \in I$;
- sort the item classes i according to non decreasing values of a_i ($i \in I$) (for the sake of simplicity, assume these are the first $|I|$ item classes);
- set $i^* = \min \{i : \sum_{j=1}^i a_j d_j > W\}$ (if no such i^* exists then $\rho = \sum_{i=1}^m d_i$);
- set $\rho = \sum_{i=1}^{i^*-1} d_i + \lfloor \frac{W - \sum_{i=1}^{i^*-1} a_i d_i}{a_{i^*}} \rfloor$.

We introduce three groups of variables. The first one contains two binary variables $\varphi_{i,r}$ and $\varphi_{i,r}^-$, indicating whether at least one item of class i , respectively rotated item class i , ($i = 1, \dots, m$), is in strip r ($r = 1, \dots, \varrho$). The second group of variables contains two integer variables $\gamma_{i,r}$ and

$\gamma_{i,r}^-$, which denote the number of items of class i , respectively rotated item class i ($i = 1, \dots, m$), to be cut from strip r ($r = 1, \dots, \varrho$). Finally the third group contains continuous variables y_r representing the width of strip r ($r = 1, \dots, \varrho$). A possible MILP model for 2DKP reads:

$$\max \quad \tau := \sum_{i \in I} \pi_i^* \sum_{r=1}^{\varrho} (\gamma_{i,r} + \gamma_{i,r}^-) \quad (7)$$

$$\text{subject to} \quad \sum_{r=1}^{\varrho} y_r \leq W \quad (8)$$

$$\sum_{i \in I} (l_i \gamma_{i,r} + w_i \gamma_{i,r}^-) \leq L \quad (r = 1, \dots, \varrho) \quad (9)$$

$$\sum_{r=1}^{\varrho} (\gamma_{i,r} + \gamma_{i,r}^-) \leq \tilde{d}_i \quad (i \in I) \quad (10)$$

$$w_i \varphi_{i,r} \leq y_r, \quad l_i \varphi_{i,r}^- \leq y_r \quad (i \in I, r = 1, \dots, \varrho) \quad (11)$$

$$\gamma_{i,r} \leq \hat{d}_i \varphi_{i,r}, \quad \gamma_{i,r}^- \leq \check{d}_i \varphi_{i,r}^- \quad (i \in I, r = 1, \dots, \varrho) \quad (12)$$

$$y_r \geq 0 \quad (r = 1, \dots, \varrho) \quad (13)$$

$$\gamma_{i,r}, \gamma_{i,r}^- \in \mathbb{Z}^+ \quad (i \in I, r = 1, \dots, \varrho) \quad (14)$$

$$\varphi_{i,r}, \varphi_{i,r}^- \in \{0, 1\} \quad (i \in I, r = 1, \dots, \varrho) \quad (15)$$

where, for $i \in I$, the coefficient $\tilde{d}_i$ represents the maximum number of items of class i in the bin, and coefficients $\hat{d}_i$ and $\check{d}_i$ denote the maximum number of items, and, respectively, of rotated items of class i , in a strip:

$$\tilde{d}_i = \min\{d_i, \lfloor \frac{(L \ W)}{(l_i \ w_i)} \rfloor\}, \quad \hat{d}_i = \min\{d_i, \lfloor \frac{L}{l_i} \rfloor\}, \quad \check{d}_i = \min\{d_i, \lfloor \frac{L}{w_i} \rfloor\} \quad (16)$$

Objective function (7) maximizes the profit of the selected items. The width feasibility of the bin is provided by constraint (8), while constraints (9) ensure that the length of each strip does not exceed the length of the bin. For rotated items, the length corresponds to the width. Constraints (10) ensure that the number of items of class i does not exceed the maximum number $\tilde{d}_i$ of items in the bin. Constraints (11) ensure that the width y_r of strip r is not smaller than the width of any item and the length of any rotated item, respectively, in the strip. Constraints (12) impose the activation of binary variables $\varphi_{i,r}$ and $\varphi_{i,r}^-$ whenever at least one item of class i and, respectively, of its corresponding rotated class, is used in strip r .

We add some valid inequalities to strengthen the continuous relaxation of the model. To this aim we introduce two coefficients θ_{min} and θ_{max} (17) denoting the ratio between the bin length and the maximum and minimum dimension of an item, respectively. Constraints (18) impose that a maximum number of different copies of items can be simultaneously present in a strip, according to the items and strip length, where ε is a small number, set to 0.1 in our computational experiments. For example, suppose that two items are available with lengths $l_1 = 3$ and $l_2 = 2$, and that strip r has length $L = 5$ (we ignore rotation for simplicity). When $\theta = 1$, we have

that $\gamma_{1,r} \leq 1$, that is, only one copy of item 1 can fit into strip r , thus forbidding, e.g., the fractional solution where $\gamma_{1,r} = 5/3$. When $\theta = 2$, we have that $\gamma_{1,r} + \gamma_{2,r} \leq 2$, that is, only 2 items, chosen among 1 and 2, can fit into strip r , etc.

Geometric constraints (19) ensure that the global area of the items in a strip does not exceed the area of the strip. To reduce the symmetry of the model, constraints (20) impose that strips have non decreasing widths. Finally, because of dominance considerations, constraints (21) impose that, for a given item class, a strip cannot contain copies of a rotated and a non-rotated item at the same time.

$$\theta_{min} = \left\lfloor \frac{L}{\max_{i \in I} \{l_i, w_i\}} \right\rfloor, \quad \theta_{max} = \left\lfloor \frac{L}{\min_{i \in I} \{l_i, w_i\}} \right\rfloor \quad (17)$$

$$\sum_{i \in I} \lfloor l_i / (L / (\theta + 1)) - \varepsilon \rfloor \gamma_{i,r} + \sum_{i \in I} \lfloor w_i / (L / (\theta + 1)) - \varepsilon \rfloor \gamma_{i,r}^- \leq \theta \quad (r = 1, \dots, \varrho, \quad \theta = \theta_{min}, \dots, \theta_{max}) \quad (18)$$

$$\sum_{i \in I} (l_i w_i) (\gamma_{i,r} + \gamma_{i,r}^-) \leq y_r L \quad (r = 1, \dots, \varrho) \quad (19)$$

$$y_r \leq y_{r+1} \quad (r = 1, \dots, \varrho - 1) \quad (20)$$

$$\varphi_{i,r} + \varphi_{i,r}^- \leq 1 \quad (i \in I, \quad r = 1, \dots, \varrho) \quad (21)$$

3.2.1 Model M2

To conclude this section we report model M2, which involves integer variables x_{ik} denoting the number of items of type i ($i = 1, \dots, m$) in strip k ($k = 1, \dots, \alpha_i$), and binary variables q_k ($k = 1, \dots, \bar{n}$) denoting whether a strip k is used (where $\alpha_i = \sum_{s \leq i} d_s$, and $\bar{n}$ is the total number of items). We refer the reader to Lodi and Monaci [9] for details.

$$\max \quad \tau := \sum_{i=1}^m \bar{p}_i \left(\sum_{k=1}^{\alpha_i} x_{ik} + \sum_{k=\alpha_{i-1}+1}^{\alpha_i} q_k \right) \quad (22)$$

$$\text{subject to} \quad \sum_{k=1}^{\alpha_i} x_{ik} + \sum_{k=\alpha_{i-1}+1}^{\alpha_i} q_k \leq ub_i \quad (i = 1, \dots, m) \quad (23)$$

$$\sum_{i=\beta_k}^m \bar{w}_i x_{ik} \leq (W - \bar{w}_{\beta_k}) q_k \quad (k = 1, \dots, \bar{n}) \quad (24)$$

$$\sum_{k=1}^{\bar{n}} \bar{l}_{\beta_k} q_k \leq L \quad (25)$$

$$\sum_{s=k}^{\alpha_i} x_{is} \leq ub_i - (k - \alpha_{i-1}) \quad (i = 1, \dots, m; k \in [\alpha_{i-1} + 1, \alpha_i]) \quad (26)$$

$$0 \leq x_{ik} \leq d_i \quad \text{integer} \quad (i = 1, \dots, m; k \in [1, \alpha_i]) \quad (27)$$

$$q_k \in \{0, 1\} \quad (k = 1, \dots, \bar{n}) \quad (28)$$

3.3 Heuristic Pattern Generation Scheme

In this section we discuss a heuristic procedure for solving 2DKP, i.e. for generating patterns, without resorting to the solution of a MILP, which can be a time consuming task. The procedure, which does not guarantee optimality of the solution, is based on the solution of two nested (NP-Hard) Bounded Knapsack Problems (BKP). The inner one combines items together into feasible strips, while the outer one deals with groups of strips. Let define ω as the number of possible different strip widths in a bin with dimensions (L, W) . This coefficient is equal to the number of different item widths (and lengths, when rotation is allowed) which are not larger than the bin width W . For each strip r ($r = 1, \dots, \omega$) let w_r be the width of the strip and define G_r as the set which includes the item classes $i \in I$ with $w_i \leq w_r$ (or $l_i \leq w_r$ for rotation) and $l_i \leq L$ (or $w_i \leq L$ for rotation). Using variables $\gamma_{i,r}$ and $\bar{\gamma}_{i,r}$ defined in Section 3.2, the inner BKP, which maximizes the profit of the items cut in a strip r for the given bin, reads:

$$\max \quad \Pi_r := \sum_{i \in G_r} \pi_i^* (\gamma_{i,r} + \bar{\gamma}_{i,r}) \quad (29)$$

$$\text{subject to} \quad \sum_{i \in G_r} (l_i \gamma_{i,r} + w_i \bar{\gamma}_{i,r}) \leq L \quad (30)$$

$$0 \leq \gamma_{i,r} \leq \hat{d}_i \quad (\gamma_{i,r} \in \mathbb{Z}^+ \quad i \in G_r) \quad (31)$$

$$0 \leq \bar{\gamma}_{i,r} \leq \check{d}_i \quad (\bar{\gamma}_{i,r} \in \mathbb{Z}^+ \quad i \in G_r) \quad (32)$$

Given the optimal solution $\gamma_{i,r}^*$ and $\bar{\gamma}_{i,r}^*$ of model (29)-(32), its corresponding value Π_r^* is set as profit of strip r in the outer BKP, and $\gamma_{i,r}^*$, $\bar{\gamma}_{i,r}^*$ are interpreted as parameters. We introduce an integer variable β_r , which represents the number of strips of type r (i.e. having width equal to

w_r) in the cutting pattern. The outer BKP, for the given bin, reads:

$$\max \quad \delta := \sum_{r=1}^{\omega} \Pi_r^* \beta_r \quad (33)$$

$$\text{subject to} \quad \sum_{r=1}^{\omega} w_r \beta_r \leq W \quad (34)$$

$$0 \leq \beta_r \leq \min(\lfloor W/\omega_r \rfloor, \min_{i \in G_r}(\lfloor \tilde{d}_i / (\gamma_{i,r} + \bar{\gamma}_{i,r}) \rfloor)) \quad (\beta_r \in \mathbb{Z}^+ \quad r = 1, \dots, \omega) \quad (35)$$

Optimally solving in sequence the two BKPs, clearly does not ensure the global optimality, because solving to optimum the inner BKP may lead to sub optimal solutions for the outer one. In addition, some patterns could be characterized by a number of items of class i greater than the corresponding demand d_i . This fact deteriorates the lower bound provided by the optimal solution of MP. We face this problem by introducing a further phase for removing the exceeding item quantities from the cutting patterns once the procedure is concluded.

As an alternative, one can consider patterns characterized by a number of items of each class i greater than the corresponding demand d_i . On one hand, this would make the solution of MP easier, because the above described procedure for generating columns would become exact (by removing the limitations on the number of available items). On the other hand, not only the lower bound provided by the optimal solution of MP could be deteriorated, but also the quality of the sub-optimal integer solutions which can be obtained with this choice might be deteriorated, in the cases in which they contain patterns exceeding the items demand. Both BKP models are solved with a dynamic programming algorithm having complexity $O(\sum_{j=1, \dots, \bar{n}} \bar{d}_j)$, where $\bar{d}_j$ represents the demand of item j . The algorithm is an extension of the dynamic programming approach for the 0-1 Knapsack Problem (see Martello and Toth [11]) and is described in detail in Furini [4].

4 Column Generation Heuristic

To solve to optimality MP given by model (1), (2) and (4), a classical column generation scheme is applied. In detail, RMP is initialized with the columns corresponding to a heuristic solution. Then the current RMP is solved to optimality, thus obtaining a vector of dual variables π^* . A *slave problem* SP is defined for every bin class j ($j = 1, \dots, n$). The SPs are heuristically solved with the heuristic pattern generation scheme (see Section 3.3) and new columns with negative reduced cost are added to the current RMP. If no column with negative reduced cost is found, the SPs are tackled with the exact model described in Section 3.2, which can find columns to be added to the current RMP or prove the optimality of the current fractional solution. In detail, we solve for each bin class the 2DKP exact model, using the dual variables π^* as profits; if the solution of the model has, for at least one bin class, negative reduced cost the corresponding column is added to the current RMP and RMP is re-optimized. Otherwise, i.e. if no negative reduced cost column is found, the current (possibly fractional) solution of RMP is optimal for the MP.

In order to obtain an integer solution, we tested three different heuristic strategies:

1. The first strategy, whose pseudo-code is given in Figure 2, consists in applying an Integer Linear Programming (ILP) solver (namely, CPLEX 12.1 [16]) to model (1)–(3), by considering all the variables used to optimally solve its continuous relaxation, i.e., the MP (1), (2) and (4). This strategy takes full advantage of the advanced routines embedded in the ILP solver.
2. The second strategy, whose pseudo-code is given in Figure 3, is a so-called *diving* heuristic, and consists in iteratively solving MP by generating columns, and then rounding-up the lower bound of a fractional variable to the closest integer, until an integer solution is obtained. In detail, after having solved to optimality MP, at each iteration we set the lower bound (*lb*) of every integer variable to the current value of the variable, and for the variable with the largest fractional part, we set the lower bound to its rounded-up value. Then we call the column generation scheme to solve the current MP with the modified lower bounds associated with the variables. Note that MP is optimally solved only at the root node, so as to obtain a valid lower bound. At the subsequent iterations, the columns are generated by applying the heuristic procedure described in Section 3.3. This method is equivalent to a Branch-and-Price algorithm, based on a depth-first strategy, and stopped when the first integer solution is found. The advantage of this method is that the column generation is not stopped at the root node of the Branch-and-Bound tree.
3. The third strategy, whose pseudo-code is given in Figure 4, combines the previous two: we execute the diving heuristic, and then we apply the ILP solver to problem (1)–(3) where we consider all the variables generated during the diving heuristic.

A time limit of 5 seconds is imposed when solving problem (1)–(3) in strategies 1 and 3.

The idea of generating a subset of variables containing at least one feasible solution, and then solving the corresponding restricted model with an ILP solver (strategies 1 and 3), can be applied to many problems with an exponential number of variables. However, to make the method of practical interest, several decisions must be taken, depending on the considered problem and on the purpose of the algorithm. If one is not interested in obtaining a strong lower bound, there is no reason to optimally solve the MP, and the column generation can be stopped before convergence. In addition, for some problems, like for example the Vertex Coloring Problem, the number of columns needed to obtain the optimal solution of MP can be huge, and thus heuristic methods (instead of applying an ILP solver) can be more suited for obtaining integer solutions. For the application of similar methods to Vertex Coloring Problems see, e.g., Malaguti [10].

5 Computational Experiments

All the algorithms described in the previous sections were implemented in C and run on one core of an Intel Core 2 Quad 2.50 GHz, with 7.7 GB shared memory, under Linux Ubuntu 9.04 operating system. All the Linear Programs and the Mixed-Integer Linear Programs were solved with CPLEX 12.1 [16].

We considered the 12 instances proposed by Cintra et al. [3] and publicly available from the web

begin

Initialize the set $\mathcal{S}$ with the Initial Heuristic Solution

1. Solve the MP (1), (2) and (4) by column generation, and update set $\mathcal{S}$ by adding the generated columns (cutting patterns);
 2. Solve model (1)–(3) with an ILP solver by considering the subset of variables x_s , $s \in \mathcal{S}$.
- end.**

Figure 2: Strategy 1.

begin

Initialize the set $\mathcal{S}$ with the Initial Heuristic Solution

1. **repeat**
 2. Solve the MP (1), (2) and (4) by column generation, and update set $\mathcal{S}$ by adding the generated columns (cutting patterns);
 3. **for each** x_s , $s \in \mathcal{S}$, **if** x_s is integer, set $lb(x_s) = x_s$;
 4. for the variable x_l with the largest fractional part, set $lb(x_l) = \lceil x_l \rceil$;
 5. **until** there exists a fractional variable.
- end.**

Figure 3: Strategy 2.

begin

Initialize the set $\mathcal{S}$ with the Initial Heuristic Solution

1. Apply strategy 2;
 2. Solve model (1)–(3) with an ILP solver by considering the subset of variables x_s , $s \in \mathcal{S}$.
- end.**

Figure 4: Strategy 3.

[15]. These instances are obtained by adding a random integer demand in the range $[1, \dots, 100]$ to the 2DKP instances $gcut1, \dots, gcut12$ from the OR library [17], and by defining three bin classes for each instance: the first bin class is the original one of dimensions (L, W) , the other two classes have dimensions $(1.2L, 0.8W)$ and $(1.1L, 0.9W)$, respectively. The features of the instances, which are called $gcut1d, \dots, gcut12d$, are summarized in Table 1, where the number m of item classes, the total number (*total*) of items, the original bin dimensions L and W , the minimum and maximum item lengths l_{min} and l_{max} and widths w_{min} and w_{max} are reported. For every instance, we consider the case where the items can be rotated and the case where rotation is forbidden. In the following tables, all the computing times are expressed in seconds.

5.1 Performance of the MILP Model M_0 for 2DKP

In this section we report the results obtained by solving the 2DKP model M_0 (see Section 3.2), and compare it with model M_2 proposed by Lodi and Monaci [9]. We first consider as profits associated with the items the values of the corresponding dual variables in RMP, so as to show that it is slightly better to use M_0 instead of M_2 when one has to solve to optimality SP. Then we consider a set of 2DKP instances from the literature, where model M_2 outperforms M_0 .

In Tables 2 and 3, we consider instances $gcut1d, \dots, gcut12d$, in the cases without and with rotation, respectively. We set as profit associated with each item class i the value of the corresponding dual variable π_i^* at the first iteration of the column generation procedure (similar results are obtained by considering the following iterations). The tables report the instance

<i>instance</i>	<i>m</i>	<i>total</i>	<i>L</i>	<i>W</i>	<i>l_{min}</i>	<i>l_{max}</i>	<i>w_{min}</i>	<i>w_{max}</i>
gcut1d	10	669	250	250	66	167	86	184
gcut2d	20	982	250	250	18	168	68	186
gcut3d	30	1489	250	250	63	176	71	186
gcut4d	50	2751	250	250	62	184	63	179
gcut5d	10	645	500	500	132	364	145	356
gcut6d	20	1064	500	500	132	355	134	372
gcut7d	30	1626	500	500	129	365	147	374
gcut8d	50	2363	500	500	131	362	127	374
gcut9d	10	590	1000	1000	292	673	341	688
gcut10d	20	830	1000	1000	269	730	260	742
gcut11d	30	1298	1000	1000	266	674	274	745
gcut12d	50	2081	1000	1000	254	746	269	723

Table 1: Instances

name in the first column. For every instance we consider 3 bin classes, thus we have 3 rows per instance. The value of the objective function (τ) is reported in the second column. The third and fourth columns of the tables report the percentage optimality gap of the value of the continuous relaxation LP_{M_0} of model M_0 , computed as $100(LP_{M_0} - \tau)/\tau$, and the corresponding computing time. Column 5 reports the time needed to solve model M_0 to optimality, and column 6 the number of nodes explored by CPLEX 12.1 [16]. The corresponding information for model M_2 is reported in columns 7 to 10 of the tables. The last row of the tables reports average values.

Considering the computing times required to solve models M_0 and M_2 to optimality, Table 3 shows M_0 is slightly faster in solving the SP when rotation is allowed, while Table 2 shows that the two models are equivalent in terms of computing time when rotation is not allowed. On one hand, this is somehow surprising, since the number of nodes explored by CPLEX 12.1 [16] to solve M_2 is on average one order of magnitude smaller than the number of nodes explored when solving M_0 . On the other hand, the time needed by CPLEX 12.1 [16] to solve the continuous relaxation of M_0 is much smaller than the corresponding computing time required by M_2 . This can be explained by considering that, although in the worst case the two models can have the same number of variables, that is, $O(\bar{n}m)$, on average, for the considered set of instances, M_2 has 15 times the number of variables of M_0 , and twice the number of constraints. Concerning the upper bounds provided by the continuous relaxation of the models, M_0 provides a better upper bound in 24 cases over 36 when rotation is not allowed, while it provides a weaker upper bound in all cases but 1 when rotation is allowed. In practice, when it comes to solve the model during the column generation, choosing model M_0 or model M_2 is equivalent.

In Tables 4 and 5 we compare the two models on the set of weighted 2DKP instances considered in [9]. The problems were solved with a time limit of 60 seconds, and the columns report the same information given in the previous tables. On this set of instances, we observe that M_2 outperforms M_0 , solving all the problems with a smaller average computing time, while M_0 cannot solve instance Hchl2 with rotation within the time limit (the corresponding remaining gap is 2.24%). Thus, we can conclude that for this second set of instances, where on average M_2 has only 5 times the number of variables of M_0 , and half the number of constraints, it is better

instance	τ	M0				M2			
		$gapLP$	T_{LP}	T_{MIP}	nodes	$gapLP$	T_{LP}	T_{MIP}	nodes
gcut1d	90000	72.073	0	0.01	5	77.329	0	0.01	5
	105000	41.963	0	0	0	50.971	0	0.01	0
	120000	28.097	0	0	0	29.411	0	0.01	0
gcut2d	96145.8	22.172	0	0.02	14	22.078	0	0.01	0
	86041.7	35.107	0	0.02	97	35.054	0	0.01	11
	72604.2	54.747	0	0.01	106	55.214	0.01	0.02	19
gcut3d	90937.5	30.110	0	0.04	165	29.453	0.01	0.03	17
	92777.8	25.967	0	0.03	143	25.641	0.01	0.03	5
	73506.9	53.879	0.01	0.04	239	53.871	0.01	0.03	47
gcut4d	174688	43.000	0.01	0.01	19	51.200	0.03	0.04	0
	180000	46.574	0	0.02	55	45.605	0.02	0.03	0
	180000	39.350	0	0.02	8	42.207	0.02	0.02	0
gcut5d	288750	30.524	0	0	27	39.903	0.01	0.01	8
	311250	17.721	0	0.01	0	24.605	0	0	0
	332083	17.854	0	0.01	32	20.433	0	0.01	9
gcut6d	330833	45.642	0	0.02	96	46.449	0	0	0
	367500	26.728	0	0.01	5	27.037	0.01	0.01	0
	350417	35.782	0	0.02	60	37.109	0	0.01	0
gcut7d	496250	61.312	0	0.01	0	62.442	0.01	0.01	9
	495000	58.608	0.01	0.01	5	57.643	0.01	0.01	0
	495000	57.269	0	0.01	8	61.550	0.01	0.01	0
gcut8d	353542	24.195	0	0.08	463	24.181	0.02	0.04	40
	311736	35.130	0	0.07	196	35.327	0.02	0.07	48
	364861	19.097	0	0.05	155	19.174	0.02	0.03	0
gcut9d	1320000	33.508	0	0.01	8	32.418	0	0	0
	1155000	44.251	0	0.01	9	45.789	0	0.01	0
	1245000	38.973	0.01	0	5	39.108	0.01	0	0
gcut10d	1980000	19.619	0	0.01	7	18.477	0	0	0
	1651670	35.333	0	0.02	33	36.875	0	0	0
	1490000	55.915	0	0.01	27	56.011	0.01	0.01	0
gcut11d	1591250	13.185	0	0.04	43	16.564	0.01	0.03	16
	1529170	9.675	0	0.01	25	16.545	0.01	0.04	30
	1490000	22.232	0	0.03	95	23.266	0.01	0.06	34
gcut12d	1570000	39.459	0	0.03	182	38.750	0.02	0.03	15
	1485000	40.408	0	0.01	14	41.114	0.02	0.02	0
	1320000	64.211	0	0.03	114	63.528	0.02	0.02	0
<i>averages</i>		37.213	0.00	0.02	68.33	38.954	0.01	0.02	8.69

Table 2: Comparison of models M_0 and M_2 with dual variables as profits. Rotation is not allowed.

instance	τ	M0				M2			
		$gapLP$	T_{LP}	T_{MIP}	nodes	$gapLP$	T_{LP}	T_{MIP}	nodes
gcut1d	81875	33.446	0	0.02	28	29.089	0.01	0.01	8
	81875	32.112	0	0.01	15	27.988	0.01	0.01	0
	75625	38.696	0	0.02	70	34.984	0.01	0.01	0
gcut2d	101667	30.465	0.01	0.21	523	22.299	0.02	0.03	0
	101458	29.427	0.02	0.2	389	21.394	0.02	0.03	24
	91145.8	39.704	0.01	0.12	161	31.261	0.02	0.03	0
gcut3d	86562.5	22.410	0	0.13	505	21.852	0.06	0.12	21
	85000	23.413	0	0.1	404	22.946	0.07	0.19	80
	85000	19.674	0	0.07	168	19.509	0.06	0.11	3
gcut4d	91666.7	24.102	0	0.36	1265	23.861	0.1	0.29	102
	95000	18.551	0	0.13	419	18.328	0.11	0.38	0
	90000	21.344	0.01	0.11	370	21.146	0.09	0.35	0
gcut5d	333333	21.026	0	0.04	131	19.862	0.01	0.02	0
	311875	24.179	0	0.03	129	23.221	0.01	0.02	17
	338125	18.117	0	0.03	51	17.037	0.01	0.02	13
gcut6d	355000	14.849	0	0.05	151	14.594	0.02	0.03	0
	317500	23.277	0	0.06	326	23.108	0.02	0.05	34
	330000	22.314	0	0.05	189	22.068	0.02	0.04	2
gcut7d	370625	48.986	0	0.05	96	48.829	0.04	0.04	0
	371250	42.786	0	0.02	64	42.690	0.04	0.04	0
	371875	47.000	0	0.05	157	46.860	0.03	0.06	26
gcut8d	506250	14.612	0	0.03	27	13.309	0.13	0.33	7
	416250	33.817	0	0.12	219	32.665	0.12	0.35	52
	403681	42.296	0	0.19	575	40.773	0.13	0.44	29
gcut9d	1490000	19.239	0	0.02	32	17.310	0.01	0.02	0
	1320000	29.211	0	0.01	19	27.565	0.01	0.01	6
	1455000	20.886	0	0.02	35	19.031	0.01	0.02	7
gcut10d	1980000	20.761	0	0.03	129	18.477	0.02	0.02	8
	1635000	40.393	0	0.04	164	38.271	0.02	0.05	53
	1490000	58.869	0	0.05	262	56.011	0.02	0.03	23
gcut11d	1732500	33.930	0	0.08	257	35.721	0.07	0.14	79
	1980000	17.023	0	0.01	4	15.511	0.07	0.12	0
	1980000	20.680	0	0.04	126	17.944	0.07	0.18	67
gcut12d	1505000	33.346	0.01	0.33	793	33.192	0.08	0.12	81
	1485000	29.737	0.01	0.08	295	27.170	0.08	0.16	31
	1490000	33.342	0	0.17	598	33.303	0.08	0.18	51
<i>averages</i>		29.000	0.00	0.09	254.06	27.199	0.05	0.11	22.89

Table 3: Comparison of models M_0 and M_2 with dual variables as profits. Rotation is allowed.

instance	τ	M0				M2			
		<i>gapLP</i>	T_{LP}	T_{MIP}	nodes	<i>gapLP</i>	T_{LP}	T_{MIP}	nodes
HH	10689	16.44	0.00	0.01	4	16.44	0.00	0.01	1
2	2535	15.19	0.00	0.43	1009	13.53	0.00	0.02	92
3	1720	17.86	0.00	0.17	640	16.59	0.00	0.01	40
A1	1820	19.38	0.00	0.26	765	17.97	0.00	0.02	62
A2	2315	17.71	0.00	0.20	988	17.08	0.00	0.03	141
STS2	4450	8.20	0.00	3.74	12827	7.16	0.00	0.40	740
STS4	9409	5.94	0.00	1.45	7522	5.69	0.00	0.16	243
CHL1	8360	10.02	0.00	0.60	1244	9.42	0.01	0.26	663
CHL2	2235	13.02	0.00	0.04	312	10.68	0.00	0.01	28
CW1	6402	12.32	0.00	0.04	163	11.67	0.00	0.03	16
CW2	5354	12.25	0.00	0.12	566	12.26	0.01	0.02	18
CW3	5287	15.14	0.00	0.10	414	12.82	0.01	0.06	43
Hchl2	9630	6.92	0.00	10.74	30213	5.79	0.02	3.32	3362
Hchl9	5100	5.57	0.00	1.03	1433	4.37	0.00	0.12	176
<i>averages</i>		12.57	0.00	1.35	4150	11.53	0.00	0.32	402

Table 4: Comparison of models M_0 and M_2 on weighted instances from [9]. Rotation is not allowed.

to use model M_2 . Belov and Scheithauer [2] came to the same conclusion when comparing a model similar to M_0 (i.e., a model where the width of the strips is a variable) with M_2 .

5.2 Performance of the Overall Method

In this section we report the results obtained by the overall method, which is compared with the heuristic algorithm proposed by Cintra et al. [3] on the *gcut1d*, ..., *gcut12d* instances. The latter algorithm is based on column generation to solve MP, and columns are generated through dynamic programming. Given a fractional solution to MP, an iterative reduction of the problem size through rounding-down and fixing of fractional variables is implemented. At each iteration, the current fractional solution is rounded-down and the corresponding items are removed from the problem. The residual problem is then solved again through column generation, and the procedure is iterated until an integer solution is produced.

The results of Cintra et al. were obtained on a PIV at 1.8 GHz with 512 MB under Linux operating system; CLP (COIN-OR LP Solver) [14] was used as LP solver. We tested our code on a similar computer and, according to our computational experience, this computing system is about 3 times slower than that used in our experiments.

The results of Cintra et al. [3] and those found by the algorithms proposed in Section 4 are reported in Tables 6 and 7, for the case without and with rotation, respectively. The first column of the tables reports the name of the instance, followed by the lower bound (LB) in column 2. Given a fractional solution to MP, LB can be obtained by rounding-up the solution value to the closest integer value corresponding to a positive integer combination of the bin areas. For the set of considered instances, the LB obtained by solving MP equals the one obtained in [3], although this latter bound could be weaker in theory, since it is computed by solving a MP including columns that can exceed the item demand. However, the two bounds are generally equal when

instance	τ	M0				M2			
		$gapLP$	T_{LP}	T_{MIP}	nodes	$gapLP$	T_{LP}	T_{MIP}	nodes
HH	11301	10.13	0.00	0.05	172	10.13	0.01	0.02	2
2	2791	4.62	0.00	1.17	1516	4.26	0.00	0.05	16
3	1900	7.41	0.00	0.55	1479	6.76	0.02	0.04	14
A1	1980	9.97	0.00	0.51	1041	9.35	0.02	0.10	252
A2	2515	8.35	0.00	0.47	1143	7.86	0.01	0.13	165
STS2	4610	4.45	0.01	12.83	34904	4.35	0.03	0.53	414
STS4	9663	3.16	0.00	10.22	20356	3.12	0.02	1.17	876
CHL1	8720	5.48	0.02	10.06	13167	5.23	0.03	1.52	944
CHL2	2354	7.31	0.00	0.36	772	6.50	0.01	0.03	73
CW1	6746	6.59	0.00	0.26	639	6.59	0.02	0.24	298
CW2	5488	9.80	0.00	0.71	2257	9.80	0.04	0.29	506
CW3	5398	12.77	0.00	0.69	1218	11.86	0.06	0.48	349
Hchl2	9921	3.78	0.01	tl	-	3.57	0.04	17.98	7042
Hchl9	5240	2.75	0.01	26.03	59150	2.68	0.03	0.59	226
<i>averages</i>		6.90	0.00	4.92	10601	6.58	0.02	1.66	798

Table 5: Comparison of models M_0 and M_2 on weighted instances from [9]. Rotation is allowed.

the demand of the items is not very small (which is the case of the considered instances), and thus it is unlikely to have patterns containing more items than the amount available. Clearly, the integer solutions later produced by our approach never contain patterns exceeding the item demands, which instead might be the case for the solutions found by the algorithm proposed by Cintra et al. [3], which iteratively reduces the problem size (i.e., the item demands).

Columns 3 to 5 report results from [3], namely the percentage optimality gap (Gap), the number of generated columns (Col) and the overall computing time (T). In Tables 6 and 7, the optimality gap for a solution of value UB is computed as $100(UB - LB)/UB$.

Columns 6 and 7 report the percentage optimality gap and the number of columns of the Initial Heuristic (the corresponding computing time is negligible). Columns 8 and 9 report the number of columns generated to solve to optimality the Master Problem (1), (2) and (4) (including the columns generated by the Initial Heuristic) and the corresponding computing time. Columns 10 and 11 report the percentage optimality gap and the global computing time of the first strategy. Columns 12, 13 and 14 report the percentage optimality gap, the number of columns (including the columns generated to solve MP to optimality) and the global computing time of the second strategy (diving procedure). Columns 15 and 16 report the percentage optimality gap and the global computing time of the third strategy. The last line of the tables reports average values.

By considering the three heuristic strategies described in Section 4, the tables show that the best performing ones are strategies 1 and 3. Strategy 1 considers the columns needed to optimally solve MP, defined by (1), (2) and (4), while strategy 3 considers in addition the columns generated by using strategy 2 (diving). In both cases, the MP integer counterpart (1)–(3) is solved by CPLEX12.1 with the considered columns and a time limit of 5 seconds of computing time. The average percentage optimality gap is 0.208% for both strategies in the case without rotation and 0.104% and 0.094% for strategies 1 and 3, respectively, in the case with rotation. The diving

instance	LB	Cintra et al. [3]			Heur.Ini.			Root		Heur1		Heur2		Heur3	
		Gap	Col	T	Gap	Col	T	Col	T	Gap	T	Gap	Col	Gap	T
gcut1d	14823125	0.384	397	0.58	8.395	16	0.02	27	0.02	0.350	0.03	0.367	28	0.350	0.29
gcut2d	16741250	0.474	492	1.31	17.386	32	0.11	64	0.11	0.205	0.19	0.810	66	0.205	0.37
gcut3d	20150000	0.583	7877	21.83	24.513	44	0.31	113	0.31	0.136	1.10	1.098	113	0.136	1.05
gcut4d	46523750	0.146	11569	60.56	21.179	72	1.10	224	1.10	0.095	6.16	0.275	228	0.105	6.38
gcut5d	41667500	0.852	110	0.17	15.924	16	0.04	28	0.04	0.090	0.04	0.816	28	0.090	0.05
gcut6d	77622500	0.702	539	0.96	11.105	27	0.09	51	0.09	0.242	5.24	0.957	52	0.245	5.35
gcut7d	123947500	0.250	1316	2.9	18.326	44	0.31	97	0.31	0.190	5.43	0.438	104	0.186	5.45
gcut8d	161075000	0.310	3958	23.67	12.635	74	2.60	191	2.60	0.017	4.48	0.456	192	0.017	7.01
gcut9d	130810000	0.780	86	0.12	14.647	16	0.06	25	0.06	0.688	0.88	2.278	26	0.688	1.08
gcut10d	260450000	0.776	434	0.81	19.532	30	0.24	69	0.24	0.296	2.84	1.121	72	0.296	5.39
gcut11d	303140000	0.429	6926	18.58	14.924	46	1.06	106	1.06	0.122	1.89	1.389	107	0.122	2.65
gcut12d	609520000	0.281	5452	36.65	21.040	69	2.15	186	2.15	0.059	2.45	0.748	192	0.059	3.80
averages		0.497	3263	14.01	16.634	41	0.67	98	0.67	0.208	2.56	0.896	101	0.208	3.24

Table 6: Results from Cintra et al. [3] and of the proposed algorithms, case with no rotation.

instance	LB	Cintra et al. [3]			Heur.Ini.			Root		Heur1		Heur2			Heur3	
		Gap	Col	T	Gap	Col	T	Col	T	Gap	T	Gap	Col	T	Gap	T
gcut1d	13828125	0.583	416	0.67	11.421	16		30	0.08	0.127	0.08	0.981	30	0.09	0.127	0.10
gcut2d	15432500	0.271	4616	37.05	20.659	33		95	2.77	0.113	7.92	0.640	98	2.82	0.069	7.87
gcut3d	19311250	0.651	12159	45.33	23.306	45		129	2.27	0.165	7.36	1.544	133	2.45	0.181	7.50
gcut4d	44767500	0.307	21902	166.68	23.034	73		239	11.87	0.017	12.26	0.300	245	12.70	0.017	14.33
gcut5d	40087500	0.736	341	0.74	19.719	16		32	0.22	0.087	0.23	0.711	34	0.25	0.062	0.26
gcut6d	70840000	0.455	2411	5.49	17.088	31		71	0.35	0.106	0.79	0.773	75	0.47	0.106	0.80
gcut7d	114820000	0.429	13326	56.78	27.780	45		131	1.68	0.065	3.08	0.414	137	2.12	0.061	3.50
gcut8d	152635000	0.508	28128	394.05	18.091	72		231	13.70	0.100	18.79	0.636	232	15.51	0.097	20.57
gcut9d	119570000	1.229	756	1.14	25.424	16		30	0.17	0.301	0.21	1.213	32	0.22	0.251	0.28
gcut10d	247880000	0.557	1545	5.68	21.289	31		82	0.86	0.056	0.91	0.896	83	1.12	0.056	1.18
gcut11d	286980000	0.854	23447	290.64	22.580	41		141	5.07	0.024	10.13	1.512	143	5.65	0.035	10.77
gcut12d	562900000	0.311	28565	690.59	18.772	69		214	14.75	0.089	19.87	0.613	217	17.04	0.069	22.19
averages		0.574	11468	141.24	20.764	41		119	4.48	0.104	6.80	0.853	122	5.04	0.094	7.45

Table 7: Results from Cintra et al. [3] and of the proposed algorithms, case with rotation.

procedure (strategy 2) obtains larger percentage gaps: 0.898% and 0.854% in the cases without and with rotation, respectively. However, this procedure in principle does not require the use of a sophisticated ILP solver as CPLEX 12.1 [16], and has very short computing times. In order to better investigate the performance of strategies 1 and 3 with respect to the time limit given to CPLEX12.1, we performed additional experiments with 15, 30 and 60 seconds of computing time. The results are summarized in Figures 5, where we report bars denoting the average percentage optimality gap for different values of the time limit, for the case without rotation in Figure 5 (a) and with rotation in Figure 5 (b). In any pair of bars, the first one corresponds to strategy 1, and the second one to strategy 3. As one can observe, by allowing more computing time it is possible to obtain very slight improvements, corresponding to better solutions found by CPLEX12.1.

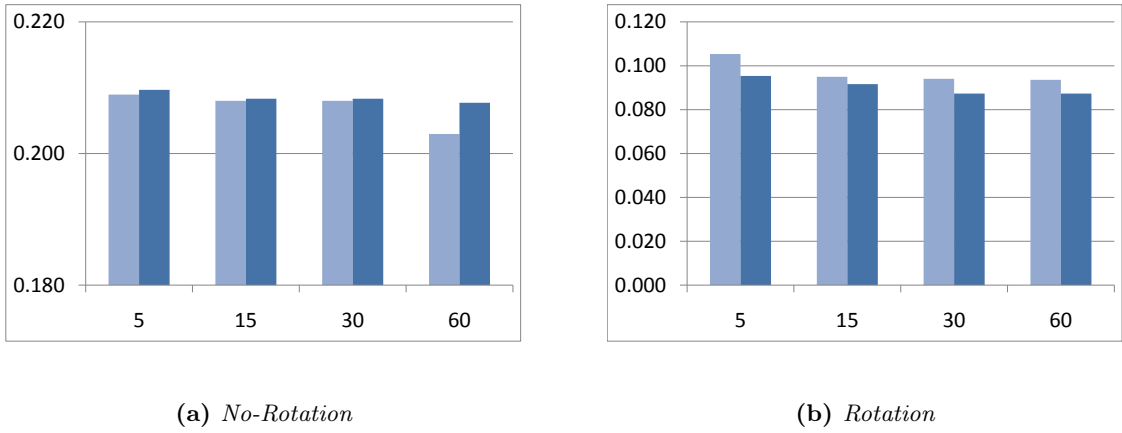


Figure 5: Percentage optimality gap as a function of the time limit.

A second issue we investigated is the sensitivity of the proposed algorithms to an increase or a reduction of the demands associated with the items. Thus, we performed additional experiments by considering the 50%, 75%, 125% and 150% of the demand associated with each item class. The results are summarized in Figures 6, where we report bars denoting the average percentage optimality gap for different values of the demand, for the case without rotation in Figure 6 (a) and with rotation in Figure 6 (b). In any group of three bars, the first one corresponds to strategy 1, the second one to strategy 2 and the third one to strategy 3. As one can observe, when the average demand is increased, the optimality gap is reduced. This is not surprising, since for large demands we can expect the value of the optimal integer solution to be closer to the value of the MP solution.

By considering the results reported by Cintra et al. [3], having average percentage gaps of 0.497% and 0.574% in the cases without and with rotation, respectively, we observe that the corresponding solution values are always worse than those obtained by strategies 1 and 3 proposed in this paper. Considering that we use an approximately three times faster computer, our computational effort is 1.5 smaller in the case with no rotation, and about 6 times smaller in the case with rotation. In particular, we observe that the proposed methods are able to optimally solve MP and to produce final improved solutions by generating a much smaller number of

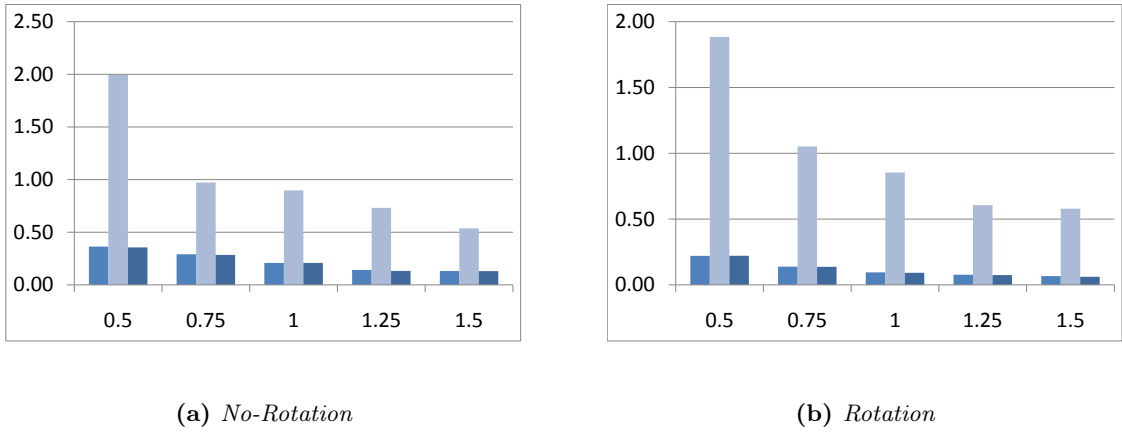


Figure 6: Percentage optimality gap as a function of the demand.

columns.

6 Conclusions

We considered a Two-Dimensional Cutting Stock Problem where stock of different sizes is available, and a set of rectangular items has to be obtained through two-staged guillotine cuts. This NP-hard problem arises in the industry whenever a sheet of paper, wood, glass, or metal has to be cut.

Starting from the classical cutting stock model of Gilmore and Gomory [7], we proposed a column generation based heuristic algorithm, and tested three different strategies to obtain integer solutions. The column generation scheme requires as subproblem the solution of a Two-Dimensional Knapsack Problem with two-staged guillotine cuts. For this Knapsack Problem, we proposed a heuristic algorithm based on dynamic programming, as well as an exact Mixed Integer Linear Programming (MILP) model.

Computational experiments were performed on a set of instances from the literature to evaluate the performance of the proposed algorithms. The MILP model was compared with a well-known compact model from the literature, and the results showed that it behaves reasonably well when used to solve the subproblems which arise during the column generation phase.

The overall heuristic algorithms obtains very small percentage optimality gaps and outperforms the most effective algorithm from the literature, producing improved solutions in smaller computing times.

7 Acknowledgments

All the authors were partially supported by MIUR, Italy. Rosa Medina Durán was partially supported by a Beca Presidente de la República - CONICYT, Government of Chile, 2008. The authors would like to thank Andrea Lodi and Michele Monaci for stimulating discussions, and

three anonymous referees for very useful comments, which helped in improving the presentation and the results as well.

References

- [1] R. Alvarez-Valdes, A. Parajon, and J.M. Tamarit. A computational study of lp-based heuristic algorithms for two-dimensional guillotine cutting stock problems. *OR Spektrum*, 24:179–192, 2002.
- [2] G. Belov and G. Scheithauer. Models with variable strip widths for two-dimensional two-stage cutting. *Tech. Rep. MATH-NM-17-2003, Dresden University*, 2003.
- [3] G.F. Cintra, F.K. Miyazawa, Y. Wakabayashi, and E.C. Xavier. Algorithms for two-dimensional cutting stock and strip packing problems using dynamic programming and column generation. *European Journal of Operational Research*, 191:61–85, 2008.
- [4] F. Furini. Decomposition and Reformulation of Integer Linear Programming Problems. *Ph.D. Dissertation, University of Bologna*, 2011.
- [5] P.C. Gilmore and R.E. Gomory. A linear programming approach to the cutting stock problem. *Operations Research*, 9:849–859, 1961.
- [6] P.C. Gilmore and R.E. Gomory. A linear programming approach to the cutting stock problem – Part II. *Operations Research*, 11:863–888, 1963.
- [7] P.C. Gilmore and R.E. Gomory. Multistage cutting stock problems of two and more dimensions. *Operations Research*, 13:94–120, 1965.
- [8] A. Lodi, S. Martello, and M. Monaci. Two-dimensional packing problems: a survey. *European Journal of Operational Research*, 141:241–252, 2002.
- [9] A. Lodi and M. Monaci. Integer linear programming models for 2-staged two-dimensional knapsack problems. *Mathematical Programming*, 94:257–178, 2003.
- [10] E. Malaguti. The Vertex Coloring Problem and Its Generalizations. *JOR*, 7:101–104, 2009.
- [11] S. Martello and P. Toth. *Knapsack Problems: Algorithms and Computer Implementations*. John Wiley & Sons, Chichester, 1990.
- [12] J. Riehme, G. Scheithauer, and J. Terno. The solution of two-stage guillotine cutting stock problems having extremely varying order demands. *European Journal of Operational Research*, 91:543–552, 1996.
- [13] G. Wäscher, H. Haussner, and H. Schumann. An improved typology of cutting and packing problems. *European Journal of Operational Research*, 183:1109–1130, 2007.
- [14] COIN-OR Linear Program Solver. An Open Source code for Solving Linear Programming Problems, <http://www.coin-or-org/Clp/index.html>.

- [15] Cutting Stock with Multiple Stock Size instances.
<http://www.ime.usp.br/~yw/BinPackingData/>.
- [16] IBM-CPLEX. <http://www-01.ibm.com/software/integration/optimization/cplex-optimizer/>
- [17] OR-Library. <http://people.brunel.ac.uk/~mastjjb/jeb/info.html>.